Proyecto Grupal 2: Ray Tracing Paralelo con DLP

Demostración 1 — CE 4302 Arquitectura de Computadores II

Randall Bryan Bolaños López
Escuela de Ingeniería en Computadores
Instituto Tecnológico de Costa Rica
Semestre I, 2026

Christopher Rodriguez
Escuela de Ingeniería en Computadores
Instituto Tecnológico de Costa Rica
Semestre I, 2026

Andres Molina
Escuela de Ingeniería en Computadores
Instituto Tecnológico de Costa Rica
Semestre I, 2026

***Resumen*—Este documento presenta la propuesta del Proyecto Grupal 2 para el curso CE 4302 Arquitectura de Computadores II. Se selecciona *ray tracing* como problema objetivo por su paralelismo de datos inherente: el cómputo por píxel es independiente. Se describen la descomposición del dominio (tiles), los diagramas para las implementaciones secuencial, SIMD/AVX2 en CPU y CUDA en GPU, y las métricas de desempeño para su evaluación.**

I. Problema Seleccionado

I-A. Descripción

El *ray tracing* es un algoritmo de síntesis de imagen que simula el transporte de luz mediante el trazado de rayos desde una cámara virtual hacia una escena tridimensional. Para cada píxel (x, y) de la imagen de resolución $W \times H$, se lanza al menos un rayo primario $\mathbf{r}(t) = \mathbf{o} + t\,\mathbf{d}$, donde $\mathbf{o}$ es el origen y $\mathbf{d}$ la dirección normalizada. El rayo se intersecta con primitivas geométricas (esferas, triángulos) y, al detectarse una colisión, se evalúa un modelo de iluminación (p. ej. Phong o Lambertian) y se emiten rayos secundarios de sombra o reflexión.

I-B. Justificación de la Selección

El *ray tracing* es un problema **embarazosamente paralelo** (*embarrassingly parallel*) por las siguientes razones técnicas:

1. **Independencia de datos por píxel**: el cálculo del color del píxel (x_i, y_i) no depende del color calculado en (x_j, y_j) para $i \neq j$. No existen dependencias de datos inter-píxel en la etapa de trazado primario.
2. **Estructura uniforme de datos**: todos los rayos aplican exactamente las mismas operaciones aritméticas sobre distintos datos de entrada (dirección del rayo). Esto se mapea directamente al modelo SIMD: *Single Instruction, Multiple Data*.
3. **Alta intensidad aritmética**: las operaciones dominantes son productos escalares, raíces cuadradas y evaluaciones de intersección con esferas ($|\mathbf{d}|^2 t^2 + \ldots = 0$), todas altamente vectorizables.
4. **Escalabilidad natural**: al incrementar la resolución (1920×1080, 3840×2160) o el número de muestras por píxel (anti-aliasing), el volumen de trabajo crece sin cambiar la estructura de dependencias, permitiendo analizar escalabilidad de forma controlada.
5. **Compatibilidad con GPU**: el modelo de ejecución SIMT (*Single Instruction, Multiple Threads*) de CUDA mapea directamente un hilo por píxel, maximizando la ocupación del dispositivo.

II. Implementación SIMD en CPU (AVX2)

II-A. Justificación de AVX2

El procesador base AMD Ryzen 5 3600 (6 núcleos / 12 hilos) soporta AVX2 (256 bits), permitiendo operar sobre **8 valores `float32` simultáneamente**. No se considera AVX-512 porque no está disponible en esta plataforma, por lo que AVX2 es el máximo nivel SIMD explotable.

II-B. Diagrama de Pipeline SIMD

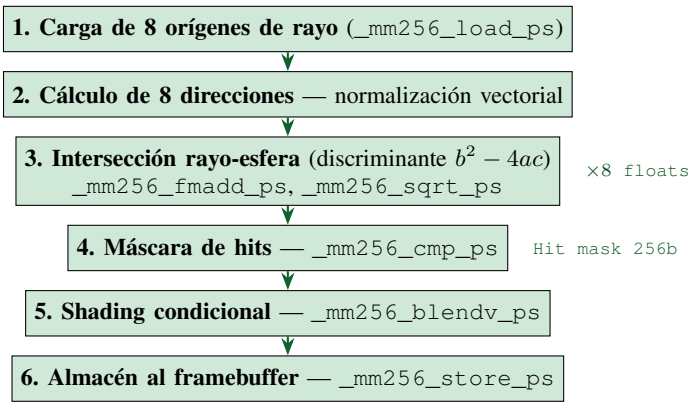


Figura 1: Pipeline de ejecución SIMD/AVX2: 8 rayos procesados en paralelo por ciclo de vectorización.

III. Implementación en GPU (CUDA)

III-A. Mapeo de Hilos

En CUDA se lanza un kernel con un hilo por píxel, organizado en bloques 2D de 16×16 hilos (256 hilos/bloque), lo que favorece la coalescencia de accesos a memoria global.

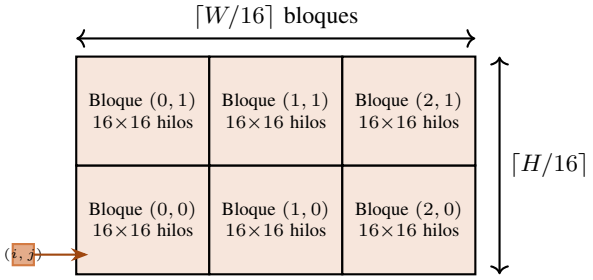


Figura 2: Organización de hilos CUDA: grid 2D de bloques 16×16, un hilo por píxel.

III-B. Diagrama de Flujo del Kernel CUDA

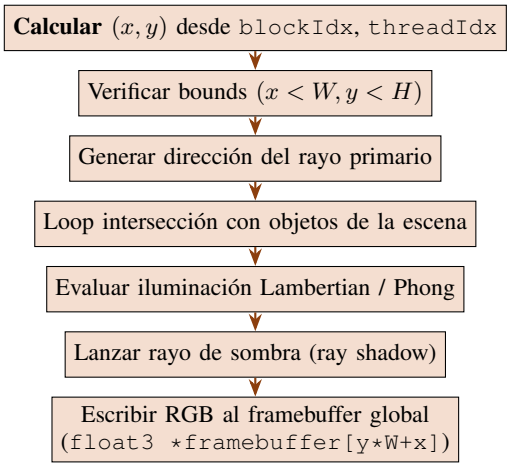


Figura 3: Flujo de ejecución del kernel CUDA de ray tracing.

IV. Métricas de Desempeño

IV-A. Métricas Seleccionadas y Justificación

La Tabla I presenta las métricas elegidas junto con la justificación técnica para el sistema objetivo.

Cuadro I: Métricas de desempeño y herramientas de medición

Métrica	**Herramienta**	**Justificación**
Tiempo total de ejecución (ms)	`chrono`/CUDA Events	Base para speedup; comparación directa entre versiones.
Speedup $S{=}T_{\text{seq}}/T_k$	Cálculo derivado	Ganancia real de paralelismo vs. baseline secuencial.
Eficiencia $E{=}S/p$	Cálculo derivado	Fracción del potencial teórico aprovechado.
Utilización SIMD (%)	`perf stat`, VTune	Valida emisión correcta de AVX2 por el compilador.
Ocupación GPU (%)	Nsight Compute	Verifica uso de SMs; $< 50\,\%$ requiere tuning de bloques.
Ancho de banda (GB/s)	Nsight Systems	Cuello de botella potencial en resoluciones >1080p.
Caché L1/L2 misses	`perf stat`	Guía ajuste de tamaño de tile $B_w \times B_h$.

V. Conclusiones de la Propuesta

El *ray tracing* es una elección técnicamente sólida para explorar Data-Level Parallelism: su independencia de datos por píxel lo hace ideal para SIMD y GPU, su intensidad aritmética aprovecha las unidades vectoriales AVX2 del Ryzen 5 3600, y la escalabilidad natural del problema permite estudiar el comportamiento de las métricas en función del tamaño de entrada. La descomposición por dominio espacial (tiles) provee una granularidad controlable tanto para el nivel SIMD como para el nivel de bloques CUDA.

Referencias

[1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.
[2] P. Shirley, *Ray Tracing in One Weekend*, 2020. [Online]. Available: https://raytracing.github.io
[3] NVIDIA Corporation, *CUDA C++ Programming Guide*, v12.4, 2024. [Online]. Available: https://docs.nvidia.com/cuda/
[4] Intel Corporation, *Intel Intrinsics Guide*, 2023. [Online]. Available: https://www.intel.com/content/www/us/en/docs/intrinsics-guide/
[5] AMD Inc., *Software Optimization Guide*, 2023.